

A CORDIC-Friendly FFT Architecture

Mohammed A. El-Motaz, Omar A. Nasr, and Karim Osama

Department of EECE Electronics and Electrical Communication Engineering Cairo University, Giza, Egypt.

Email: mohammed_elmotaz@yahoo.com, omarnasr@ieee.org and karim.central@gmail.com.

Abstract—Fast Fourier Transform (FFT) is one of the basic building blocks in signal processing and communications systems. The butterflies-based structure of the FFT is the main reason for the reduced number of arithmetic operations required to implement the transform. From implementation point of view, the complex rotations used in butterflies can be implemented by using COordinate Rotation Digital Computer (CORDIC). This implementation strategy reduces the hardware complexity compared to the direct implementation of the butterflies using complex multipliers.

In this paper, we introduce a restructure of the butterflies of the radix-2 FFT to be more CORDIC friendly. This algorithm-level modification of the FFT is friendly towards all CORDIC types, including those introducing non-fixed gain. Compared with the conventional radix-2 FFT algorithm, the proposed algorithm introduces a substantial increase in performance. For example, it achieves superior signal to quantization noise ratio (SQNR), with around 14 dB gain for 8 to 1024 points FFT. In addition, in pipeline architectures the modification leads to an improvement in latency or a reduction in the total area, with an improvement in either of 38% for 1024 points FFT.

Keywords—COordinate Rotation Digital Computer (CORDIC), Fast Fourier Transform (FFT).

I. INTRODUCTION

The CORDIC algorithm is an iterative method used to implement a variety of trigonometric and transcendental functions in hardware. The CORDIC algorithm was first described by Jack E. Volder [1]. It uses shift and add operations to perform complicated arithmetic operations that would otherwise require multiplications and extensive use of look-up tables [2]. The most commonly used operation performed by CORDIC is vector rotation. Hence, CORDIC is used as an efficient way to realize multiplication-free FFT.

Several algorithms have been introduced to modify the original CORDIC algorithms for systems involving fixed angle rotations. [3] presents optimization schemes and CORDIC circuits for fixed and known rotations. In [4], MVR-CORDIC was introduced to improve speed and SQNR. However it resulted in unequal CORDIC gain at every FFT stage, forcing gain compensation after each micro-rotation. In [5], EEAS-CORDIC was introduced to increase the angle space and simplify the scaling stage slightly. In [6], MSR-CORDIC integrates the scaling stage into the rotation stage, which resulted in an improvement in the speed of the algorithm. However, there was a significant decrease in the SQNR. The unequal CORDIC gain and large Read Only Memory (ROM) sizes required by the modified CORDICs, made the original CORDIC algorithm the best choice for authors implementing CORDIC-based FFT, [7], [8], [9] and [10].

Some authors have proposed a scaling-free CORDIC algorithm. This technique and its modified versions, as described in [11], [12], [13] and [14], are based on choosing elementary angle subset from the whole CORDIC angle space to synthesize the rotation angle. The condition, this subset is chosen based on, is that $\cos(\theta_{elementary}) = 1$ relative to operands' fixed point representation. Since the CORDIC scaling factor is related to the cosine of the angle, the algorithm is scaling-free. Although it seems that it has solved the unequal CORDIC gain problem, however, this algorithm does not take the advantage of the whole CORDIC angle domain; so it needs more iterations to reach the same SQNR as in the other CORDIC types.

Hence, compared with the conventional CORDIC, the scaling-free algorithm can be useful for systems having no information about the rotation angle. However, the algorithm is not hardware friendly for systems involving known angle rotations like FFT.

All the modifications in [4], [5] and [6] focus on the CORDIC algorithm. Recent researches focus on the optimization of the implementation of FFT architecture as in [7], [8], [9] and [10]. In this work, we propose a modification to the FFT algorithm that removes the need for CORDIC gain compensation after each FFT stage. Hence, not only scaling-free CORDIC algorithms can be used to implement the FFT, but also other more efficient CORDIC algorithms such as [4], [5] and [6]. This introduces a gain in the speed of the algorithm, while, at the same time, guaranteeing a superior SQNR.

The remainder of this paper is organized as follows. Section II and III review the related CORDIC techniques and the general hardware architectures required to implement FFT. Section IV introduces the proposed radix-2 FFT algorithm. Section V and VI present the results in the form of SQNR simulation measurements for different CORDIC techniques and comparison of the hardware performance between the original and the novel FFT algorithm. section VII draws the conclusion.

II. REVIEW OF RELATED CORDIC TECHNIQUES

CORDIC essentially considers the problem of rotating a complex number $x + jy$ to a new point $x' + jy'$ by angle θ . The rotation can be computed as

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}. \quad (1)$$

In the following, We show how the different CORDIC techniques synthesize the complex rotation operation.

III. REVIEW OF RELATED CORDIC TECHNIQUES

CORDIC essentially considers the problem of rotating a complex number $x + jy$ to a new point $x' + jy'$ by an angle θ . The rotation can be computed as

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}. \quad (2)$$

In the following, We show how the different CORDIC techniques synthesize the complex rotation operation.

A. Conventional CORDIC

The previous equation can be written as follows,

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \cos(\theta) \begin{bmatrix} 1 & -\tan(\theta) \\ \tan(\theta) & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}. \quad (3)$$

Jack E. Volder suggested to decompose θ to discrete set of angles, $(\theta_0, \theta_1, \dots, \theta_n, \dots, \theta_{N-1})$ where $\theta_n = \tan^{-1}(2^{-n})$. Therefore, the previous equation will be

$$\begin{bmatrix} x'_n \\ y'_n \end{bmatrix} = CG(n) \begin{bmatrix} 1 & -d(n)2^{-n} \\ d(n)2^{-n} & 1 \end{bmatrix} \begin{bmatrix} x'_{n-1} \\ y'_{n-1} \end{bmatrix} \quad (4)$$

$$CG(n) = \cos(\tan^{-1}(2^{-n})) = \frac{1}{\sqrt{1+2^{-2n}}} \quad (5)$$

where $d(n) = \pm 1$. So $x' + jy'$ can be evaluated by,

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \prod_{n=0}^{N-1} CG(n) \begin{bmatrix} 1 & -d(n)2^{-n} \\ d(n)2^{-n} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (6)$$

where N is the number of iterations. N usually equals to the register size to maintain acceptable SQNR level. As $N \rightarrow \infty$, $\prod_{n=0}^{N-1} CG(n) = 0.6073$. The multiplicative inverse of 0.6073, equals to 1.647, is called "CORDIC gain". This number is independent of the angle. Which means that the CORDIC gain is the same for all angles. Most systems do not need to compensate for this gain; therefore, it does not pose any problem.

Thus, the conventional CORDIC algorithm reduces to a number of shift-add operations which are very attractive to hardware designers. The shift in every iteration is determined by $d(n)$ and it can be stored in a ROM for known rotations, as in FFT case. This simple form of CORDIC is valid for any angle, but it always requires N iterations, regardless of the angle of rotation. Although fixing the number of rotations simplify the design of the CORDIC, it results in a large overhead because the choice of the number of iterations is usually based on the worst case of all possible angles supported by the CORDIC.

B. MVR-CORDIC

In [4], the authors proposed that in some applications, where there is some information about the angle of rotation known beforehand, the number of iterations can be reduced. So we can perform the rotation using only K iteration, a subset of the full N iterations. The following equations describe the algorithm,

$$\begin{bmatrix} x'_n \\ y'_n \end{bmatrix} = CG(n) \begin{bmatrix} 1 & -d(n)2^{-t(n)} \\ d(n)2^{-t(n)} & 1 \end{bmatrix} \begin{bmatrix} x'_{n-1} \\ y'_{n-1} \end{bmatrix} \quad (7)$$

$$CG(n) = \frac{1}{\sqrt{1+2^{-2t(n)}}} \quad (8)$$

where $t(n) \in [0, N-1]$. So $x' + jy'$ can be evaluated by,

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \prod_{n=0}^{M-1} CG(n) \begin{bmatrix} 1 & -d(n)2^{-t(n)} \\ d(n)2^{-t(n)} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (9)$$

where $M < N$. The parameters of MVR-CORDIC, $d(n)2^{-t(n)}$, are chosen to minimize the rotation residual error. The same SQNR, as conventional CORDIC, can be achieved using a smaller number of iterations because it is allowed to skip some micro-rotations and repeating others. But while performing the rotation in fewer iterations, this method introduces unequal CORDIC gain, which depends on the angle of rotation. Hence, this gain must be compensated with a factor that reverses the effect of this gain. This compensation was not necessary in the conventional CORDIC.

C. EEAS-CORDIC

A method was proposed in [5] increases angle space by constructing linear combinations of shifts and adds to realize $\tan(\theta)$ as follows,

$$\begin{bmatrix} x'_n \\ y'_n \end{bmatrix} = CG(n) \begin{bmatrix} 1 & -SPT_I(n) \\ SPT_I(n) & 1 \end{bmatrix} \begin{bmatrix} x'_{n-1} \\ y'_{n-1} \end{bmatrix} \quad (10)$$

$$CG(n) = \frac{1}{\sqrt{1 + \left(\sum_{i=1}^K d_i(n)2^{-t_i(n)} \right)^2}} \quad (11)$$

$$SPT_I(n) = \sum_{i=1}^K d_i(n)2^{-t_i(n)} \quad (12)$$

where $t(n) \in [0, N-1]$. So $x' + jy'$ can be evaluated by,

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \prod_{n=0}^{M-1} CG(n) \begin{bmatrix} 1 & -SPT_I(n) \\ SPT_I(n) & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (13)$$

Where $M < N$. The parameters of EEAS-CORDIC, $SPT_I(n)$, are chosen to minimize the rotation residual error. This method is faster than MVR-CORDIC and has a larger angle space, but it still introduces unequal gains. Hence, a compensation stage is needed. Even though [5] proposed a method to compensate for the CORDIC gain using the same EEAS-CORDIC structure, called "ET-II" scaling operation, it is still cumbersome.

D. MSR-CORDIC

In [6], the authors proposed to mix the scaling phase with the rotation phase in order to remove the need for compensation as follows,

$$\begin{bmatrix} x'_n \\ y'_n \end{bmatrix} = CG(n) \begin{bmatrix} SPT_R(n) & -SPT_I(n) \\ SPT_I(n) & SPT_R(n) \end{bmatrix} \begin{bmatrix} x'_{n-1} \\ y'_{n-1} \end{bmatrix} \quad (14)$$

$$CG(n) = \frac{1}{\sqrt{\left(\sum_{j=1}^L d_j(n)2^{-r_j(n)} \right)^2 + \left(\sum_{i=1}^K d_i(n)2^{-t_i(n)} \right)^2}} \quad (15)$$

$$SPT_R(n) = \sum_{j=1}^L d_j(n)2^{-r_j(n)} \quad (16)$$

$$SPT_I(n) = \sum_{i=1}^K d_i(n)2^{-t_i(n)} \quad (17)$$

where $t(n)$ and $r(n) \in [0, N-1]$. So $x' + jy'$ can be evaluated by,

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \prod_{n=0}^{M-1} CG(n) \begin{bmatrix} SPT_R(n) & -SPT_I(n) \\ SPT_I(n) & SPT_R(n) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (18)$$

where $M < N$. The MSR-CORDIC parameters are chosen in a way to have almost unity gain, hence, it removes the need for gain compensation. However, it has a detrimental effect on the SQNR of the result for the same number of iteration; since the parameters of MSR-CORDIC, $SPT_I(n)$ and $SPT_R(n)$, are chosen to minimize both the rotation and gain residual error, not only rotation residual error as in the previous techniques.

IV. FFT ARCHITECTURE

Since the rotation angles used in the computations are known beforehand in the FFT, these rotations can be computed using the CORDIC algorithm, which provide the same throughput as a complex multiplier while using less silicon area [16]. The coefficients required by the CORDIC algorithm can be pre-calculated and stored in a ROM.

As in [15] and [16], ASIC pipeline FFT architectures are divided, in general, into parallel-pipeline and serial-pipeline implementations. Parallel-pipeline architecture achieves the maximum throughput, minimum latency, but it has the maximum area compared to other architectures. Figure 1 shows parallel-pipeline architecture where BF, PDC and DL denote to Butterfly block, pipeline dedicated CORDIC block and delay line block, respectively.

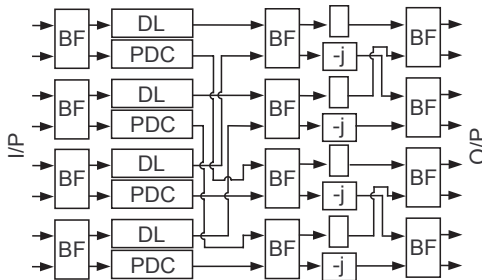


Fig. 1. CORDIC parallel-pipeline FFT architecture

Serial-pipeline architecture achieves high throughput, smaller area, but relatively high latency. Figure 2 shows Radix-2 Single-Path Delay Feedback (R2SDF) serial-pipeline architectures. There is another type of serial-pipeline architecture called Radix-2 Multiple-Path Delay Commutator (R2MDC); but this type is not usually used since it has low memory utilization.

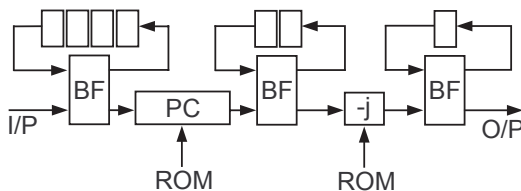


Fig. 2. CORDIC R2SDF FFT architecture.

A comparison between the complexity of different architectures using conventional radix-2 FFT is shown in table I, where T_{PC} and A_{PC} are the number of clocks and adders required to perform a CORDIC iteration, respectively. Throughput is calculated assuming the same critical path.

To avoid performing constant multiplication after each CORDIC rotation, FFT implementations usually choose the conventional CORDIC or MSR-CORDIC [6]. Therefore, an algorithmic level solution is proposed in the following section to allow using other more efficient CORDIC types, even if their CORDIC gains depends on the angle of rotation.

V. PROPOSED RADIX-2 FFT

The proposed algorithm suggests to reorganize the rotations in the conventional radix-2 FFT such that the two butterfly operands are multiplied by twiddle factors having the same CORDIC gain. The reorganization procedure is done according to the following observations,

- the sign of an angle does not affect the CORDIC gain as in equations (8), (11) and (15). Therefore, CORDIC gain is always the same for both a certain angle and its negative;
- Butterfly operation is a linear operation. Figure (3, a) shows a butterfly with its original twiddle factors: $W_{up} = 1$ and $W_{down} = e^{j\phi}$. Figure (3, b) shows the modified twiddle factors: $W_{up} = Ke^{j\theta}$ and $W_{down} = Ke^{j(\phi+\theta)}$. These twiddle factors were chosen to maintain the same difference of the old twiddle factors in Figure (3, a). Hence, due to the CORDIC linearity, the output of (b) will equal to the output of (a), except of a fixed complex scale factor;
- multiplying all the points of Discrete Fourier Transform (DFT) in time domain by a constant is equivalent to multiplying the DFT's points in frequency domain by the same constant.

This paper presents two main approaches to explain the modified radix-2 FFT algorithm: an analytical approach using equations and a graphical approach using Signal Flow Graph (SFG). Note that, in the analytical approach, CORDIC gain is omitted for simplicity.

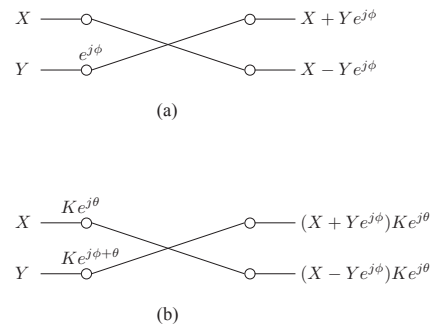


Fig. 3. Butterfly operation is sensitive only to the phase difference

In Decimation In Time (DIT), input sequence $x(n)$ is divided into smaller sub-sequences. With dividing it into even

TABLE I
Hardware performance comparison between different FFT architectures

FFT Architecture	Latency (clk)	Throughput (symbol/clk)	Adders (count)
Parallel-pipeline	$(\log_2(N) - 2)T_{PC} + \log_2(N)$	N	$\frac{N}{2}((\log_2(N) - 2)A_{PC} + 4\log_2(N))$
R2SDF	$(\log_2(N) - 2)T_{PC} + \log_2(N) + N - 1$	1	$(\log_2(N) - 2)A_{PC} + 4\log_2(N)$

and odd sequences, the DFT equation reaches to the following,

$$X(k) = \sum_{r=0}^{\frac{N}{2}-1} x(2r)W_N^{2rk} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x(2r+1)W_N^{2rk} \quad (19)$$

where $W_N^{2rk} = e^{-j2\pi \frac{2rk}{N}}$ is the twiddle factor. With $g(r) = x(2r)$ and $h(r) = x(2r+1)$, the proposed algorithm suggests to modify the twiddle factors as following,

$$X(k) = W_N^{\frac{k}{2}} \left(W_N^{-\frac{k}{2}} \sum_{r=0}^{\frac{N}{2}-1} g(r)W_N^{rk} + W_N^{\frac{k}{2}} \sum_{r=0}^{\frac{N}{2}-1} h(r)W_N^{rk} \right) \quad (20)$$

Each twiddle factor pair belonging to the next butterfly operand pair, namely W_{up} and W_{down} , is decomposed into two parts:

- **Modified Twiddle Factor (MTF):** These are the modified twiddle factors that will be applied during the butterfly operation. The twiddle factors are chosen in a way to keep the same phase difference as the original twiddle factors. The new twiddle factors are related to the old ones as follows,

$$MTF_{up} = \sqrt{\frac{W_{up}}{W_{down}}} \quad (21)$$

$$MTF_{down} = \sqrt{\frac{W_{down}}{W_{up}}} \quad (22)$$

- **Propagating Twiddle Factor (PTF):** As stated before, the new twiddle factors have the same phase difference as the old ones, hence, the output will be the same as the old output, but scaled with a complex scale factor. This complex scale factor is calculated as,

$$PTF_{up,down} = \sqrt{W_{up}W_{down}} \quad (23)$$

Figure 4 illustrates the transition of first stage in conventional 8 points radix-2 FFT to the modified one, where the circles filled with same pattern are inputs to the same butterfly in the next stage. Twiddle factors between brackets are *PTFs* and the rest are the *MTFs*. 'A' is the CORDIC gain associated with each rotation and it has the same subscript and superscript as its generating twiddle factor. Mapping equations (21), (22) and (23) to figure 4, if W_{up} and W_{down} are W_8^0 and W_8^2 in the fifth and seventh FFT nodes at the first stage, then MTF_{up} , MTF_{down} , PTF_{up} and PTF_{down} are W_8^{-1} , W_8^1 , W_8^1 and W_8^1 , respectively. *PTFs* are the same for all sub FFTs inputs, where the input of the next sub FFT are shown in the dotted rectangles in figure 4.

Each of $g(r)$ and $h(r)$ in equation (20) represents the input points for the $\frac{N}{2}$ points DFTs, and are handled as the original input $x(n)$. For example, to generate DFT output $G(k)$ of even

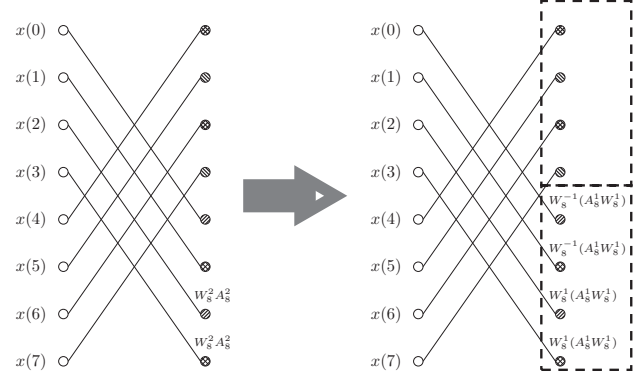


Fig. 4. The transition of first stage in conventional radix-2 FFT to modified one

input points, namely $g(r)$, it will be decomposed into,

$$G(k) = W_N^{\frac{k}{2}} \left(W_N^{-\frac{k}{2}} \sum_{p=0}^{\frac{N}{4}-1} g'(p)W_N^{pk} + W_N^{\frac{k}{2}} \sum_{p=0}^{\frac{N}{4}-1} h'(p)W_N^{pk} \right) \quad (24)$$

where $g'(p) = g(2p)$ and $h'(p) = g(2p+1)$. Figure 5 illustrates the transition of second stage toward the modified algorithm where the input of the next sub FFT are shown in the dotted rectangles.

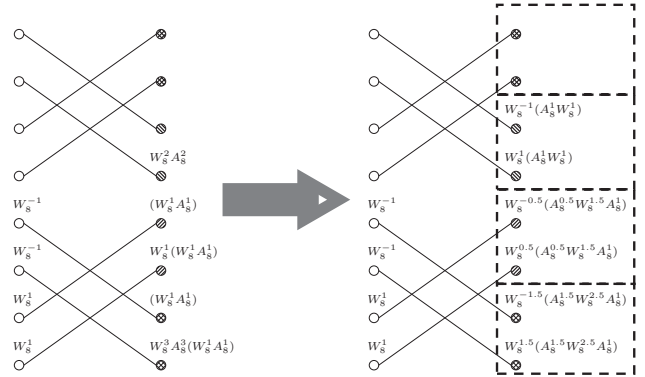


Fig. 5. The transition of second stage toward the modified algorithm

After the last stage, the scale factors are merged into one complex scale factor for each output point. This scale factor can be easily compensated. Moreover, if the FFT is used in a communications system, the scaling factors can be compensated in the channel inversion stage. Figure 6 shows complete SFG for 8 points modified radix-2 FFT.

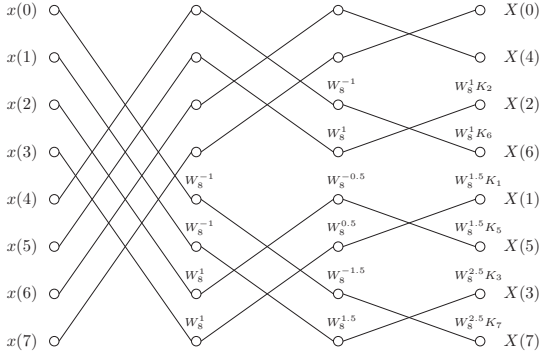


Fig. 6. 8 points modified DIT radix-2 FFT

The above analysis has not exploited all the possible modifications to the twiddle factors, since the new twiddle factors phase are constrained to an angle and its negative while keeping the old phase difference between the angles used in the butterflies. However, there maybe other alternatives to choose new twiddle factors, while maintaining the same CORDIC gain. To illustrate this, figure 7 redraw the SFG of the modified 8 point FFT by replacing the twiddle factors with the rotation angles in degree. In the first stage we have rotations by 45° and -45° whose difference equals to 90° . Those angles, namely 45° and -45° , needs one CORDIC iteration to synthesize them. However, If 45° and -45° are replaced with 0° and -90° , the following is observed,

- 1) 0° and -90° have the same CORDIC gain;
- 2) they produced $PTF_{up,down} = W_8^1$ transferred to the output of its sub FFT, shown in the dotted rectangles in figure 7;
- 3) they can be synthesized by multiplying by 1 and $-j$, which needs no CORDIC iterations.

Figure 8 illustrates the previous steps on 8 points FFT where the angles written in degrees. Generally, This will apply to larger FFT sizes. Hence, the number of complex rotations in the whole FFT without equalization is $N(\log_2(N) - 3) + 4$ instead of $N(\log_2(N) - 2) + 2$. Thus, the synthesis is more efficient.

VI. SQNR SIMULATION MEASUREMENTS

We have simulated the proposed algorithm with different CORDIC types, namely MVR-CORDIC [4], EEAS-CORDIC [5] and MSR-CORDIC [6], and calculate their SQNR. In the following we describe the simulation environment:

- 90° , -90° , and 180° degree rotations do not need any CORDIC operation.
- A Mixture of Dynamic Programming and Branch-and-Bound is employed to find the best CORDIC rotations. However, This search is cumbersome. Therefore it is calculated offline then the results are saved in a ROM to be used on hardware.
- Equalization of scaling factor is assumed to be done without error. This equalization is either done in the

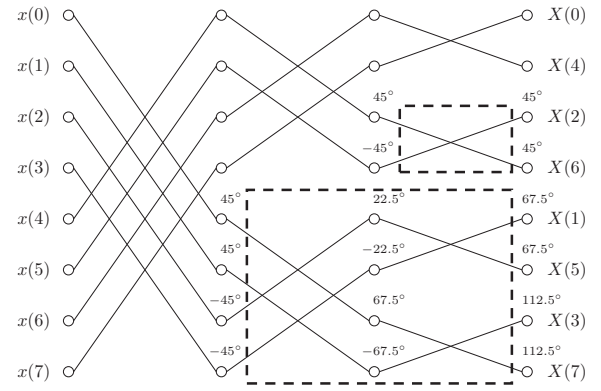


Fig. 7. Rotations of 8 point Modified Radix-2 FFT in degree

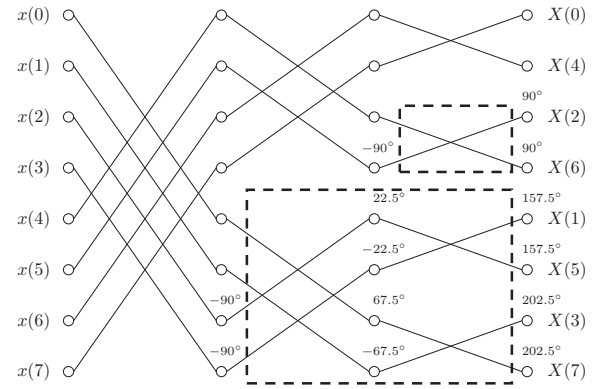


Fig. 8. First stage optimization in the Modified Radix-2 FFT

stage after FFT, such as channel equalization in communication system, or by a vector of MSR-CORDICs with high accuracy.

- The simulation results exclude round off errors and do not depend on the architecture type.
- The shifter employed in the CORDIC algorithm can shift up to 15 bits.
- The angles of rotations are chosen such that the elementary CORDIC gain is limited between $\frac{1}{3}$ and 3. This will preserve accuracy by limiting round off error when fixed point simulation is performed.
- The elements of the FFT input test vectors are iid standard complex Gaussian random variables.
- 1024 point FFT is taken as a case study.

In the following sub sections, we present the SQNR results.

A. MVR and EEAS CORDIC with the proposed algorithm

Table II shows the SQNR for the output of FFT using both MVR-CORDIC and 2-SPT EEAS-CORDIC, [4] and [5] respectively, using different number of iterations. If conventional

FFT is used with its original butterfly structure and rotation angles, constant multipliers are needed after each CORDIC to compensate for the unequal CORDIC gains, which increases the hardware complexity for the same SQNR.

TABLE II
Modified radix-2 FFT performance using MVR-CORDIC and EEAS-CORDIC

MVR-CORDIC		EEAS-CORDIC	
Iter.	SQNR(dB)	Iter.	SQNR(dB)
2	46.4	1	36.9
3	71.3	2	103.3
4	93.7	-	-

B. MSR-CORDIC with conventional and the proposed algorithm

A comparison is done between the conventional and modified radix-2 FFT using MSR-CORDIC. Table III shows the SQNR for MSR-CORDIC in 1024 point conventional and modified radix-2 FFT with different number of iterations, where N/G column denotes to Normal / Generalized which are MSR-CORDIC types, as described in [6].

TABLE III
Conventional and Modified radix-2 FFT performance using (2,1) MSR-CORDIC

Conventional FFT			Modified FFT		
N/G	Iter.	SQNR(dB)	N/G	Iter.	SQNR(dB)
N	2	39.6	N	1	41.9
G	2	40.7	G	1	44.2
N	3	71.0	N	2	113.6
G	3	79.3	-	-	-
N	4	99.6	-	-	-

VII. HARDWARE PERFORMANCE COMPARISON

For comparison, we choose (2,1) normal MSR-CORDIC to synthesize both algorithms but with 4 iterations for conventional algorithm and 2 iterations for modified algorithm. As stated in table III, They both have achieved the highest and comparable SQNR. Table IV shows hardware performance gain without equalization. There is 14 dB SQNR gain in all architectures. As shown, parallel-pipeline architecture's latency decreased by 38%. Thus, pipeline registers are also decreased by 38%. If the equalization is done in the FFT architecture, the improvement in adders count of R2SDF and latency of parallel-pipeline is 33.33%.

TABLE IV
Hardware performance comparison without equalization

Hardware parameters	Latency (clk)/Adders (count)		
	Conv. FFT	Mod. FFT	Improv.
Parallel-pipeline	42 / 86016	26 / 77856	38.1% / 9.5%
R2SDF	1065 / 168	1049 / 104	1.5% / 38.1%

VIII. CONCLUSION

In this paper, We have introduced a novel CORDIC-friendly radix-2 FFT algorithm that enables using efficient CORDIC type, even if the CORDIC type does not support

equal CORDIC gain. The algorithm reorganizes the conventional FFT rotations to unify the CORDIC gain multiplied by all butterfly operands in the whole sub FFT stage. The proposed algorithm achieves 14 dB SQNR gain, compared to conventional FFT architecture using half the number of iterations. Moreover, for 1024 points FFT, either latency or processing area are decreased by 38% in serial-pipeline and parallel-pipeline architectures.

REFERENCES

- [1] J. E. Volder, "The CORDIC trigonometric computing technique," *Electronic Computers, IRE Transactions on*, vol. EC-8, no. 3, pp. 330–334, Sept 1959.
- [2] J. S. Walther, "A unified algorithm for elementary functions," in *Proceedings of the May 18-20, 1971, spring joint computer conference*. ACM, 1971, pp. 379–385.
- [3] P. Meher and S. Park, "CORDIC designs for fixed angle of rotation," *Very Large Scale Integration (VLSI) Systems, IEEE Transactions on*, vol. 21, no. 2, pp. 217–228, Feb 2013.
- [4] C.-S. Wu and A.-Y. Wu, "Modified vector rotational CORDIC (MVR-CORDIC) algorithm and architecture," *Circuits and Systems II: Analog and Digital Signal Processing, IEEE Transactions on*, vol. 48, no. 6, pp. 548–561, Jun 2001.
- [5] C.-S. Wu, A.-Y. Wu, and C.-H. Lin, "A high-performance/low-latency vector rotational CORDIC architecture based on extended elementary angle set and trellis-based searching schemes," *Circuits and Systems II: Analog and Digital Signal Processing, IEEE Transactions on*, vol. 50, no. 9, pp. 589–601, Sept 2003.
- [6] C.-H. Lin and A.-Y. Wu, "Mixed-Scaling-Rotation CORDIC (MSR-CORDIC) algorithm and architecture for high-performance vector rotational dsp applications," *Circuits and Systems I: Regular Papers, IEEE Transactions on*, vol. 52, no. 11, pp. 2385–2396, Nov 2005.
- [7] K. Kalyani, D. Sellathambi, and S. Rajaram, "Reconfigurable FFT using CORDIC based architecture for MIMO-OFDM receivers," in *Information Communication and Embedded Systems (ICICES), 2013 International Conference on*, Feb 2013, pp. 670–675.
- [8] A. Malashri and C. Paramasivam, "Low power and memory efficient FFT architecture using modified CORDIC algorithm," in *Information Communication and Embedded Systems (ICICES), 2013 International Conference on*, Feb 2013, pp. 1041–1046.
- [9] S.-N. Tang, C.-H. Liao, and T.-Y. Chang, "An area- and energy-efficient multimode FFT processor for WPAN/WLAN/WMAN systems," *Solid-State Circuits, IEEE Journal of*, vol. 47, no. 6, pp. 1419–1435, 2012.
- [10] S. Jiangyi, T. Yinghui, W. Mingxing, and Y. Zhe, "A novel design of 1024-point pipelined FFT processor based on CORDIC algorithm," in *Intelligent System Design and Engineering Application (ISDEA), 2012 Second International Conference on*, Jan 2012, pp. 80–83.
- [11] K. Maharatna, A. S. Dhar, and S. Banerjee, "A VLSI array architecture for realization of DFT, DHT, DCT and DST," *Signal Processing*, vol. 81, no. 9, pp. 1813–1822, 2001.
- [12] A. Dhar and S. Banerjee, "An array architecture for fast computation of discrete hartley transform," *Circuits and Systems, IEEE Transactions on*, vol. 38, no. 9, pp. 1095–1098, Sep 1991.
- [13] K. Maharatna, S. Banerjee, E. Grass, M. Krstic, and A. Troya, "Modified virtually scaling-free adaptive CORDIC rotator algorithm and architecture," *Circuits and Systems for Video Technology, IEEE Transactions on*, vol. 15, no. 11, pp. 1463–1474, Nov 2005.
- [14] F. Jaime, M. Sanchez, J. Hormigo, J. Villalba, and E. Zapata, "Enhanced scaling-free CORDIC," *Circuits and Systems I: Regular Papers, IEEE Transactions on*, vol. 57, no. 7, pp. 1654–1662, July 2010.
- [15] S. He and M. Torkelson, "A new approach to pipeline FFT processor," in *Parallel Processing Symposium, 1996., Proceedings of IPPS '96, The 10th International*, Apr 1996, pp. 766–770.
- [16] E. Wold and A. Despain, "Pipeline and parallel-pipeline FFT processors for VLSI implementations," *Computers, IEEE Transactions on*, vol. C-33, no. 5, pp. 414–426, May 1984.